SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 50%

Time Duration: 1 Hour

QUESTIONS: 4 **Total Marks:40**

Annexure A

CASE STUDY

Code of Conduct:

I Nethra P *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

nethra

Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

I. If a patient has Fever AND Cough → Possible Flu

Ii. If a patient has Fever AND Rash → Possible Measles

Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia

Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) [Rule 1]
II. ∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) [Rule 2]
III. ∀x ∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) [Rule 3]
IV. Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) P1: SoilDry → IrrigationNeeded
2) P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod
3) P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk
4) Facts: SoilDry = True, CropWheat = True

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication → move ¬ inward → distribute).

(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause □. Draw the resolution proof tree.

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

1) Stench [1,2] → Wumpus is adjacent to cell [1,2]
2) Breeze [1,1] → Pit is adjacent to cell [1,1]
3) Glitter [x,y] → Gold is at [x,y]
4) ¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1 — Smart Medical Diagnosis System

(a) Propositional Logic Representation

We first define atomic propositions for every symptom and every diagnosis used in the knowledge base:

Symbol	Meaning (Proposition)
F	Patient has Fever
C	Patient has Cough
Rs	Patient has Rash
B	Patient has Breathlessness
M1	Possible Flu
M2	Possible Measles
M3	Possible Pneumonia

Using these symbols, the rules and facts of the knowledge base (KB) are written as:

Rule I: $F \wedge C \rightarrow M1$

Rule II: $F \wedge Rs \rightarrow M2$

Rule III: $C \wedge B \rightarrow M3$

Facts for Patient A:

F = True, C = True, B = True (Rs is not asserted, hence treated as unknown/False by default)

(b) Forward Chaining

Forward chaining starts from the known facts and repeatedly fires any rule whose premises are fully satisfied, adding the conclusion as a new fact, until no more rules can fire.

Step	Known Facts Checked	Rule Fired	Fact Derived
1	F = True, C = True	Rule I: $F \wedge C \rightarrow M1$	M1 (Possible Flu) is added to KB
2	F = True, Rs = unknown	Rule II: $F \wedge Rs \rightarrow M2$	Cannot fire — Rs is not established, so M2 is NOT derived
3	C = True, B = True	Rule III: $C \wedge B \rightarrow M3$	M3 (Possible Pneumonia) is added to KB

Step	Known Facts Checked	Rule Fired	Fact Derived
4	No further premises satisfied	—	Fixed point reached; algorithm halts

Final derived diagnoses for Patient A: Possible Flu (M1) and Possible Pneumonia (M3). Measles (M2) cannot be concluded because the Rash symptom was never asserted as true.

(c) Backward Chaining — Goal: 'Patient A has Pneumonia' (M3)

Backward chaining starts from the goal M3 and recursively reduces it to sub-goals until each sub-goal matches a known fact.

Step 1: Goal = M3. Search the KB for a rule whose conclusion is M3 → Rule III: C ∧ B → M3.

Step 2: M3 is reduced to two sub-goals: C (Cough) and B (Breathlessness).

Step 3: Check sub-goal C against the fact base → C = True (matches a known fact). Sub-goal C is proved.

Step 4: Check sub-goal B against the fact base → B = True (matches a known fact). Sub-goal B is proved.

Step 5: Since both sub-goals (C and B) are proved True, the parent goal M3 is proved True.

<u>**Goal Reduction Tree**</u>

```
                         M3  (Possible Pneumonia)
                          |
                   Rule III:  C ∧ B → M3
                    /              \
                  C                  B
              (Cough)           (Breathlessness)
                 |                   |
          Fact: C = True        Fact: B = True
             ✓ PROVED              ✓ PROVED
```

Conclusion: Both leaf nodes (C and B) resolve to known True facts, so the goal 'Patient A has Pneumonia' is logically confirmed by backward chaining.

Case Study 2 — Autonomous Traffic Management Agent

(a) Unification of Rule 1 with the Given Facts

Rule 1 is: ∀x Vehicle(x) ∧ EmergencyType(x) → ClearPath(x). To apply this rule to a specific vehicle we must unify its premises with matching facts in the KB.

Step	Unification Attempted	Resulting Substitution θ
1	UNIFY(Vehicle(x), Vehicle(Ambulance))	θ1 = { x / Ambulance }
2	UNIFY(EmergencyType(x), EmergencyType(Ambulance)) under θ1	Consistent with θ1 → θ2 = { x / Ambulance }
3	Combine (MGU)	θ (MGU) = { x / Ambulance }
4	Apply θ to the rule head ClearPath(x)	ClearPath(Ambulance) — new derived fact

Since both premise literals unify with a single, consistent substitution {x/Ambulance}, this is the Most General Unifier (MGU), and the rule fires to derive ClearPath(Ambulance).

(b) Resolution Proof that Proceed(CarA) is True

Step 1 — Convert all rules and facts to Conjunctive Normal Form (CNF):

Rule 1 (Vehicle(x) ∧ EmergencyType(x) → ClearPath(x)):

C1: ¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x)

Rule 2 (ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x)) — split the conjunction into two clauses:

C2: ¬ClearPath(x) ∨ GreenSignal(x)

C3: ¬ClearPath(x) ∨ ¬RedSignal(x)

Rule 3 (Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x)):

C4: ¬Vehicle(x) ∨ ¬Behind(x,y) ∨ ¬GreenSignal(y) ∨ Proceed(x)

Facts (unit clauses):

C5: Vehicle(Ambulance)   C6: EmergencyType(Ambulance)

C7: Vehicle(CarA)   C8: Behind(CarA, Ambulance)

Negated goal (for refutation):

C9: ¬Proceed(CarA)

Step 2 — Resolution steps:

#	Resolve	Unifier	Resulting Clause
1	C5: Vehicle(Ambulance) with C1: ¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x)	x/Ambulance	¬EmergencyType(Ambulance) ∨ ClearPath(Ambulance)
2	Result of (1) with C6: EmergencyType(Ambulance)	—	ClearPath(Ambulance)
3	Result of (2) with C2: ¬ClearPath(x) ∨ GreenSignal(x)	x/Ambulance	GreenSignal(Ambulance)
4	C4: ¬Vehicle(x) ∨ ¬Behind(x,y) ∨ ¬GreenSignal(y) ∨ Proceed(x) with result of (3)	y/Ambulance	¬Vehicle(x) ∨ ¬Behind(x,Ambulance) ∨ Proceed(x)
5	Result of (4) with C7: Vehicle(CarA)	x/CarA	¬Behind(CarA,Ambulance) ∨ Proceed(CarA)
6	Result of (5) with C8: Behind(CarA,Ambulance)	—	Proceed(CarA)
7	Result of (6) with C9: ¬Proceed(CarA) (negated goal)	—	□ (empty clause)

Since the resolution process derives the empty clause (□), the negation of the goal leads to a contradiction with the KB. Therefore, by refutation, Proceed(CarA) is proved to be TRUE.

(c) Sufficiency of the KB for Two Simultaneous Emergency Vehicles

The current KB is not sufficient to correctly handle two emergency vehicles approaching the same intersection at the same time. The reasons and required extensions are as follows:

- Conflict in GreenSignal: Rule 1 and Rule 2 would independently derive ClearPath(x) and GreenSignal(x) for every vehicle x that satisfies Vehicle(x) ∧ EmergencyType(x). If two emergency vehicles approach from conflicting directions, the KB would derive GreenSignal(Ambulance1) and GreenSignal(Ambulance2) simultaneously, which is physically inconsistent (two conflicting green signals at one junction).
- Missing predicate — Priority/Severity: A predicate such as Priority(x,n) or Severity(x,n) is needed so the agent can rank emergency vehicles when more than one qualifies for ClearPath at the same time.
- Missing predicate — Conflict/Direction: A predicate such as ConflictsWith(x,y) (or Direction(x,d)) is required to know which approaches physically conflict, so the KB can decide that only one of two conflicting vehicles receives GreenSignal at a time.
- New rule needed: ∀x∀y Vehicle(x) ∧ Vehicle(y) ∧ EmergencyType(x) ∧ EmergencyType(y) ∧ ConflictsWith(x,y) ∧ HigherPriority(x,y) → ClearPath(x) ∧

¬ClearPath(y). This enforces a strict, non-contradictory choice when two emergencies coincide.

- Consistency constraint: An integrity constraint of the form ¬(GreenSignal(x) ∧ GreenSignal(y) ∧ ConflictsWith(x,y)) should be added so the KB itself can detect and reject inconsistent models.

Justification: First-order logic as stated is monotonic and treats every instance of the rule independently; without an explicit priority/ordering predicate and a conflict predicate, the KB has no mechanism to prevent two simultaneously true but mutually exclusive conclusions (GreenSignal for two conflicting vehicles). Adding priority and conflict predicates, together with a rule that combines them, resolves this gap logically.

Case Study 3 — Agricultural Expert Reasoning System

(a) Conversion of P1, P2, P3 and Facts to CNF

P1: SoilDry → IrrigationNeeded

Step 1 (Eliminate implication, A → B ≡ ¬A ∨ B): ¬SoilDry ∨ IrrigationNeeded

Step 2 (No negations to move inward, no conjunctions to distribute): already in CNF.

CNF (P1): ¬SoilDry ∨ IrrigationNeeded

P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod

Step 1 (Eliminate implication): ¬(IrrigationNeeded ∧ CropWheat) ∨ ApplyDripMethod

Step 2 (Move ¬ inward using De Morgan's law): ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

Step 3 (Already a disjunction of literals — no distribution required): CNF form obtained.

CNF (P2): ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk

Step 1 (Eliminate implication): ¬(¬ApplyDripMethod ∧ CropWheat) ∨ CropAtRisk

Step 2 (Move ¬ inward, De Morgan's law): (ApplyDripMethod ∨ ¬CropWheat) ∨ CropAtRisk

Step 3 (Flatten the disjunction — already CNF, no distribution needed):

CNF (P3): ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk

Facts

CNF (Fact 1): SoilDry

CNF (Fact 2): CropWheat

(b) Resolution Refutation — Prove 'ApplyDripMethod'

We negate the goal and add it to the CNF knowledge base, then show that resolution derives the empty clause □, proving the goal is entailed.

C1: ¬SoilDry ∨ IrrigationNeeded (from P1)

C2: ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod (from P2)

C3: SoilDry (fact)

C4: CropWheat (fact)

C5: ¬ApplyDripMethod (negated goal)

#	Resolve	Resulting Clause
1	C3 (SoilDry) with C1 (¬SoilDry ∨ IrrigationNeeded)	IrrigationNeeded
2	Result of (1) with C2 (¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod)	¬CropWheat ∨ ApplyDripMethod
3	Result of (2) with C4 (CropWheat)	ApplyDripMethod
4	Result of (3) with C5 (¬ApplyDripMethod)	□ (empty clause)

<u>Resolution Proof Tree</u>

```
SoilDry        ¬SoilDry ∨ IrrigationNeeded
   \              /
     IrrigationNeeded          ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod
          \                      /
            ¬CropWheat ∨ ApplyDripMethod        CropWheat
                  \                      /
                      ApplyDripMethod        ¬ApplyDripMethod (negated goal)
                            \                  /
                                □ (empty clause)
```

Since the negated goal combined with the KB leads to a contradiction (□), the original goal ApplyDripMethod is proved TRUE by resolution refutation.

(c) Effect of Removing the Fact 'SoilDry'

Step 1: Without SoilDry as a known fact, clause C1 (¬SoilDry ∨ IrrigationNeeded) can no longer be resolved with a matching unit clause, so IrrigationNeeded can no longer be derived.

Step 2: Since IrrigationNeeded is not derivable, clause C2 (¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod) also cannot be resolved to a unit clause, so ApplyDripMethod cannot be proved TRUE.

Step 3: ApplyDripMethod is now simply 'unknown' — it has neither been proved true nor proved false by the KB. In classical (monotonic) logic, failure to prove a sentence is NOT the same as proving its negation.

Step 4: P3 requires ¬ApplyDripMethod ∧ CropWheat to conclude CropAtRisk. Under strict classical entailment, ¬ApplyDripMethod is not established (only 'not proved'), so CropAtRisk does NOT logically follow from the KB.

Step 5: However, many rule-based expert systems use the Closed World Assumption (CWA) / negation-as-failure, where any proposition that cannot be proved true is treated as false. Under CWA, since ApplyDripMethod cannot be derived, the system would treat ¬ApplyDripMethod as True, and — combined with CropWheat = True — CropAtRisk WOULD be concluded.

Conclusion: Under classical (open-world) logical entailment, CropAtRisk does not follow once SoilDry is removed, because ¬ApplyDripMethod is unproven rather than proven. Under the Closed World Assumption commonly used by rule-based expert systems, CropAtRisk does follow, since the failure to derive ApplyDripMethod is interpreted as its negation. This illustrates the difference between monotonic classical reasoning and non-monotonic, negation-as-failure reasoning typically used in practical expert systems.

Case Study 4 — Wumpus World Knowledge Agent

(a) Belief State Construction and Modus Ponens Application

Percepts received by the agent: Stench[1,2], Breeze[1,1], Glitter[2,2]. Applying Modus Ponens (from P and P→Q, infer Q) systematically to each rule:

Percept (P)	Rule Applied (P → Q)	Derived Conclusion (Q) — Modus Ponens
Stench[1,2] = True	Stench[1,2] → Wumpus adjacent to [1,2]	Wumpus is adjacent to [1,2], i.e. Wumpus ∈ {[1,1],[1,3],[2,2]}
Breeze[1,1] = True	Breeze[1,1] → Pit adjacent to [1,1]	Pit is adjacent to [1,1], i.e. Pit ∈ {[1,2],[2,1]}
Glitter[2,2] = True	Glitter[x,y] → Gold at [x,y]	Gold[2,2] — Gold is located at cell [2,2]

List of all derived conclusions: Wumpus-adjacent-to-[1,2] (candidate cells [1,1], [1,3], [2,2]); Pit-adjacent-to-[1,1] (candidate cells [1,2], [2,1]); Gold[2,2].

(b) Forward Chaining — Locating the Wumpus and the Pit

Step 1: Assert Stench[1,2] as a fact → Rule 1 fires → conclude 'Wumpus adjacent to [1,2]'. The neighbours of [1,2] are [1,1], [1,3] and [2,2], so Wumpus ∈ {[1,1],[1,3],[2,2]}.

Step 2: The agent has already safely visited/occupies [1,1] and has not perceived Stench there, so by Rule 4 (¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]) together with the absence of a stench percept at [1,1], the agent eliminates [1,1] from the Wumpus candidate set. Updated: Wumpus ∈ {[1,3],[2,2]}.

Step 3: Assert Breeze[1,1] as a fact → Rule 2 fires → conclude 'Pit adjacent to [1,1]'. The neighbours of [1,1] are [1,2] and [2,1], so Pit ∈ {[1,2],[2,1]}.

Step 4: Assert Glitter[2,2] as a fact → Rule 3 fires → conclude Gold[2,2].

Step 5: Apply Rule 4 wherever both ¬Wumpus[x,y] and ¬Pit[x,y] can be established: since no breeze was perceived at [1,2] itself in the given percepts beyond [1,1], and no additional stench percept has been reported for [2,2] or [1,3], the KB cannot yet fully resolve either candidate set to a single cell — it can only narrow the possibilities as shown above.

Summary of possible locations — Wumpus: {[1,3], [2,2]}; Pit: {[1,2], [2,1]}. Neither location is uniquely determined with the percepts given; further exploration (or the fact that the Wumpus and Gold do not usually coexist safely) would be required to fully disambiguate.

(c) Safety Evaluation of the Path [1,1] → [1,2] → [2,2]

Cell [1,1]: Confirmed safe — the agent currently occupies it without harm, so ¬Wumpus[1,1] and ¬Pit[1,1] both hold by direct observation.

Cell [1,2]: This cell itself carries the Stench percept, meaning the Wumpus is in one of its neighbouring cells, not necessarily in [1,2] itself. Since no Wumpus/Pit rule directly places the Wumpus or a Pit inside [1,2], and no breeze was reported there, [1,2] can be treated as passable — but visiting it does not by itself resolve the danger, because moving through [1,2] still leaves the Wumpus's exact cell (either [1,3] or [2,2]) unresolved.

Cell [2,2]: This is where the risk concentrates. From part (b), [2,2] remains in the unresolved Wumpus-candidate set {[1,3],[2,2]}. At the same time, [2,2] holds the Glitter/Gold percept. Because the KB has NOT eliminated [2,2] from the Wumpus candidates, Rule 4 (¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]) cannot be applied to certify Safe[2,2] — its precondition ¬Wumpus[2,2] is not established.

Conclusion: The path [1,1] → [1,2] → [2,2] is NOT provably safe with the percepts given. [1,1] and [1,2] can be traversed without contradiction, but [2,2] cannot be certified as Safe by logical inference because the Wumpus has not been ruled out of that cell — it remains a live candidate alongside [1,3]. A cautious logical agent should not proceed to [2,2] to collect the gold until it gathers an additional percept (for example, checking for Stench at [1,3] or [2,1]) that allows Rule 4 to conclusively establish ¬Wumpus[2,2], after which Safe[2,2] could be derived and the path validated.